

第三章 基于有状态任务切片的链下计算验证

链下计算验证体系的可靠运行涉及三个层面的问题。计算验证层面，需要保证链上能够以可控成本对链下执行结果进行仲裁；验证者组织层面，需要保证验证者集合在开放网络中能够被安全、公平选出；行为约束层面，需要保证验证者切实履行了验证工作。本章聚焦于第一个层面，研究面向链下计算验证的高效争议仲裁方案。现有链下计算验证方案在面对长时复杂计算任务时，仍存在争议定位效率低、链上裁决开销大、验证结果确认周期长等问题。为此，本章提出基于有状态任务切片的链下计算验证方案 CleVer，通过自适应状态切片在执行过程中持续生成中间状态快照，在此基础上设计两层切片争议定位机制和超线性累计质押协议，实现链上裁决轻量化与争议快速收敛。理论与实验分析表明，所提方案能够在保证裁决可靠性的同时，将争议处理开销约束在可控范围内，有效提升链下计算验证的时效性与争议解决效率。

3.1 引言

本节围绕链下计算验证面临的核心挑战展开分析，首先陈述现有方案在应对长时间复杂计算任务时存在的瓶颈，进一步提出本章方案需要满足的设计目标，并总结本章的工作贡献。

1. 问题陈述

链下计算验证的相关研究已取得显著进展，但当现有方案被应用于长时间执行的通用复杂计算任务时，仍存在以下三方面问题。

(1) 执行过程缺乏面向验证的结构化组织，争议定位效率受限。现有方案在执行阶段通常不主动生成服务于验证过程的中间结构，执行过程对验证而言近似为黑盒。当争议发生时，系统通常依赖多轮交互逐步逼近分歧位置，链上交互轮次随计算规模呈对数增长。由于缺乏可直接引用的结构化中间状态，争议定位难以充分利用链下已有执行信息实现快速收敛。其结果是，争议范围难以及时压缩到足够小的可裁决区间，链上裁决开销难以与任务规模有效解耦。

(2) 现有争议解决协议缺乏有效的快速终止机制，错误方可持续拖延争议过程。在典型的交互式二分争议协议中，若各轮争议的边际参与成本近似恒定，则实质错误的一方在争议推进过程中缺乏随轮次增加而持续增强的退出压力，可能持续参与直至最终裁

决，从而拉长单次争议周期。此外，在缺乏有效约束时，恶意参与者还可通过重复发起低成本挑战实施资源消耗型攻击。

(3) 事后验证模式导致错误发现滞后，纠错代价较高。当前主流方案大多采用“先执行后挑战”的事后验证模式，验证者必须等待任务执行完毕后才能开始重放检查，即使错误在执行中间阶段已经产生也无法被及时截断，错误点之后的计算资源均被浪费，纠错往往需要回退并重新执行较长区间。

综上，本章关注的核心问题是，面向长时间复杂计算，如何设计一种链下计算验证方案，使执行轨迹能够被结构化地组织并验证，争议能够在链下高效收敛、在协议层面有限终止，最终以较低的链上开销达成确定性仲裁。

2. 设计目标

围绕上述问题，本章提出以下四项设计目标，并将其作为后续方案设计、理论与实验分析的基本标准。

(1) 执行轨迹应在执行过程中被动态划分为可供验证使用的段级区间，并生成可引用的中间状态快照与承诺表示，使原本面向全局计算结果的验证转化为基于中间状态的分段检查。由此，验证者无需等待任务完成即可对已有段展开验证，验证工作能够与执行过程并行推进。

(2) 争议定位的主要收敛过程应尽可能在链下完成，链上仅承担必要且开销受限的最终裁决。争议定位不应继续依赖与任务规模相关的多轮链上交互，而应利用结构化的中间状态与链下承诺对比机制，将分歧快速收敛至可被链上裁决的最小单元，从而降低链上交互轮次与裁决负担。

(3) 争议解决协议应当在有限轮次内结束，不能被恶意参与方无限拖延。为此，协议应通过合理的经济机制对持续参与争议施加递增约束，使错误方在争议推进过程中面临不断增强的经济压力，从而尽早退出争议，缩短多数争议的平均解决路径。

(4) 链上最终裁决的执行开销应由系统参数预先设定，不受任务规模影响，单次裁决的资源消耗须控制在预算内，以确保方案在不同规模下具备良好的兼容性。

3. 本章贡献

针对上述问题与设计目标，本章提出了一种基于有状态任务切片的链下计算验证方案 CleVer，主要贡献如下。

(1) 提出一种面向通用复杂计算验证的自适应状态切片机制。针对执行过程缺乏面向验证的结构化组织问题,本章在链下执行过程中依据段预算约束动态划分执行轨迹为段级区间,在每个段边界处保存中间状态快照并向链上提交对应承诺,使任意段均可从其起始快照独立重放验证,为后续争议定位与伴随式验证提供统一的结构化基础。

(2) 提出基于两层切片的争议定位方法与基于超线性累计质押的争议解决协议。在争议定位方面,首先利用第一层快照承诺对比确定争议焦点段,再通过第二层细分将争议收敛至满足裁决阈值约束的最小裁决单元。在争议解决方面,设计超线性累计质押机制,对持续参与争议施加递增的边际博弈成本,增强错误方提前退出的经济压力。该设计使链上争议交互轮次被压缩为与任务规模无关的常数量级,并保证争议在由系统参数预先约束的有限轮次内终止。

(3) 构建伴随式验证架构以降低错误发现延迟与纠错代价。在状态切片基础上,本章构建了执行与验证并行推进的伴随式验证架构。依托执行过程中持续生成的状态快照,验证者无需等待任务完成即可对已执行区间展开检查,从而将错误发现时点前移至执行过程中,并将纠错代价限制在局部快照区间内,进而提升长时间复杂任务场景下的验证时效性与可扩展性。

3.2 链下计算有状态切片验证模型

本节为第三章的形式化基础。首先给出系统参与角色与协议运行流程,建立系统模型;随后定义计算任务、最小裁决单元等核心形式化对象;最后给出对手模型与裁决正确性要求。

3.2.1 workflow 概览

1. 角色定义

CleVer 涉及四类角色。

(1) 任务发布者。任务发布者是计算任务的发起方,通过区块链提交任务描述与系统参数,并预存任务酬劳与协议运行费用。任务发布者不参与计算或验证,其职责在于定义任务并为协议各方提供经济激励的初始资金。

(2) 执行方。执行方是链下的计算实体,负责在确定性执行环境中运行计算任务。执行方在执行过程中按段预算约束动态生成中间状态快照,并对快照计算承诺提交至区

块链，为后续验证与争议裁决提供依据。执行方在任务发布时需质押保证金，作为诚实执行的经济担保。

(3) 验证者。验证者负责对执行方提交的计算结果进行正确性监督。在 CleVer 的伴随式验证架构下，验证者可在执行方持续生成快照的过程中并行验证已完成的段，无需等待任务结束。当验证者发现某段的重放结果与执行方承诺不一致时，可在挑战窗口期内发起挑战。本章假设系统中存在至少一个诚实验证者，验证者的安全选举与审计等工作将在后续章节讨论。

(4) 链上裁决合约。链上裁决合约是部署在区块链上的智能合约程序，作为协议的可信仲裁者承担以下三项职责。第一，记录任务描述与快照承诺；第二，在争议发生时对最小裁决单元执行链上重放裁决；第三，依据裁决结果执行经济结算。链上裁决合约不参与链下执行，也不承担全流程验证任务，其裁决可信性由区块链的共识机制保障。

2. 模型运行流程

CleVer 的整体计算验证流程如图 5 所示，图中展示了任务发布、链下执行与快照生成、伴随式验证、争议定位解决以及最终经济结算五个阶段之间的衔接关系。

(1) 阶段一：任务发布。任务发布者将计算任务描述与系统参数通过链上交易提交至裁决合约。系统参数包括段预算 B 、裁决阈值 b 、挑战窗口期 T_{ch} 、累计质押曲线 $D(\cdot)$ 等，所有参数在任务上链后不可更改。

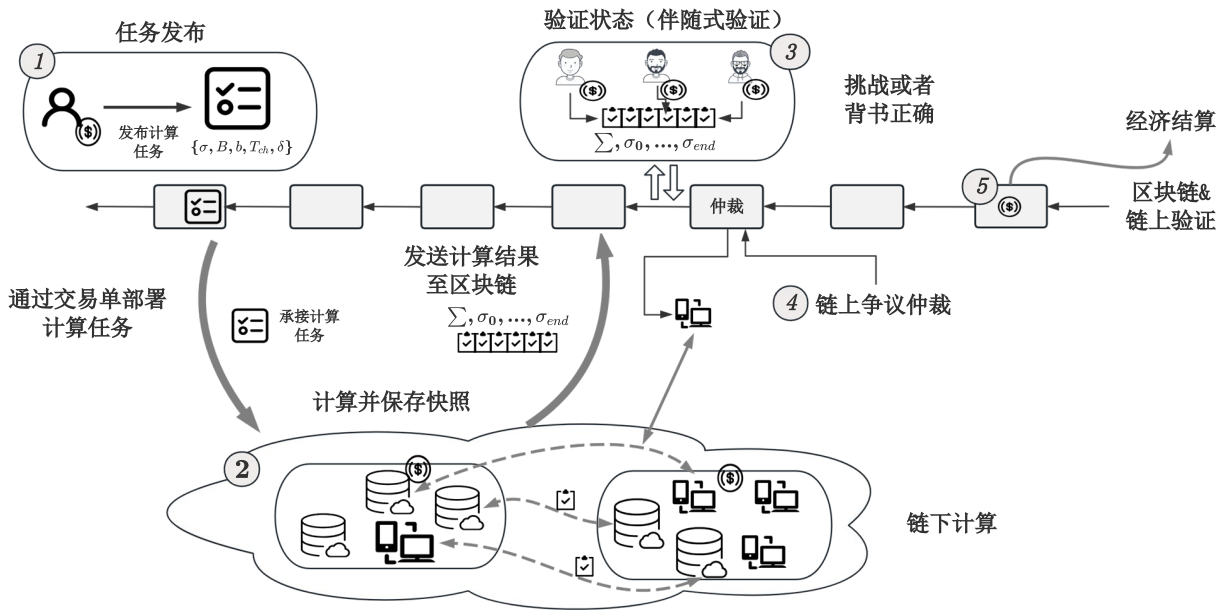


图 5 CleVer 计算验证工作流程图

(2) 阶段二：链下执行与快照生成。执行方在链下确定性环境中运行任务。执行过程中，执行方依据段预算 B 动态选择切片边界，在每个边界处保存中间状态快照，并将快照的状态承诺提交至裁决合约。快照生成与执行同步进行，链上仅记录承诺值，不涉及实际计算数据。

(3) 阶段三：伴随式验证。验证者在执行方持续提交承诺的过程中即可开始验证工作。验证者从链下获取相邻快照间的段数据，在本地重放并将结果与下一快照承诺比对。若发现不一致，验证者在挑战窗口期 T_{ch} 内向裁决合约提交挑战交易，若验证结果与承诺一致，验证者对该段结果进行正确性背书。若窗口期内无有效挑战，直接进入阶段五。

(4) 阶段四：争议定位解决。若验证者发起有效挑战，系统进入争议处理流程。系统从所有被挑战段中选取索引最小者作为争议焦点段；随后，执行方与挑战者代表各自对焦点段进行二次细分，提交细粒度状态承诺序列，之后双方进入超线性累计质押博弈，每轮需补足的质押金额按超线性函数递增；最终，若一方退出则该方判负，若双方均坚持到底则定位分歧点并触发链上重放裁决。

(5) 阶段五：结果终局化与经济结算。当挑战窗口期内不存在有效挑战，或争议裁决认定执行方正确时，系统将任务结果终局化。结果终局化后，释放执行方保证金并分配收益。争议下的败诉方损失其全部累计质押，胜诉方获得对方质押作为补偿。

3.2.2 形式化定义

本节在运行流程的基础上，给出 CleVer 核心概念的形式化定义。

定义 3.1（计算任务） 一个链下通用计算任务被建模为确定性状态转移系统

$$\mathcal{T} = (\Sigma, \sigma_0, \delta, \sigma_{\text{end}}) \quad (3.1)$$

其中 Σ 为状态空间， $\sigma_0 \in \Sigma$ 为初始状态， δ 为确定性状态转移函数， $\sigma_{\text{end}} \in \Sigma$ 为终止状态。整个计算过程对应一条有限的状态序列（执行轨迹）

$$\sigma_0 \xrightarrow{\delta} \sigma_1 \xrightarrow{\delta} \cdots \xrightarrow{\delta} \sigma_N = \sigma_{\text{end}} \quad (3.2)$$

其中 N 为总步数，输出正确终止状态记为 $y^* = \sigma_{\text{end}}$ 。

定义 3.1 刻画了正确执行的语义，给定相同的 σ_0 与 δ ，诚实执行方应产生相同的执行轨迹与终止状态。CleVer 的验证目标即为在不重复执行全部 N 步的前提下，判定执行方是否忠实地按 δ 完成了从 σ_0 到 σ_{end} 的转移。

若直接要求链上验证整条长度为 N 的执行轨迹，其开销不可接受。CleVer 通过在执行过程中周期性保存中间状态快照，将执行轨迹划分为一系列段级区间。本文假设计算任务在同构虚拟机下执行，即链上重放与链下执行遵循相同状态转移规则。

定义 3.2（第一层快照序列与段划分） 执行方在链下执行任务 $\mathcal{T}$ 时，依据段预算 B 动态生成快照序列

$$\hat{\sigma}_0, \hat{\sigma}_1, \dots, \hat{\sigma}_m \quad (3.3)$$

其中 $\hat{\sigma}_0 = \sigma_0$ 为初始状态， $\hat{\sigma}_m = \sigma_{\text{end}}$ 为终止状态。相邻快照 $(\hat{\sigma}_i, \hat{\sigma}_{i+1})$ 之间对应一个段 Seg_i ，即从 $\hat{\sigma}_i$ 出发经若干步 δ 到达 $\hat{\sigma}_{i+1}$ 的执行片段， $i = 0, 1, \dots, m-1$ 。快照序列将执行轨迹划分为 m 个段级区间，每个区间可独立重放验证。

定义 3.3（指令权重与段预算约束） 设指令权重上界函数 $w: \text{Inst} \rightarrow \mathbb{R}^+$ 将每条指令映射到一个正实数权重，反映其在链上重放时的执行开销上界。其中 Inst 为本章执行虚拟机指令集。同时对段 Seg_i 内包含的指令序列 $(e_{i,1}, e_{i,2}, \dots, e_{i,n_i})$ ，定义其累计权重

$$W(\text{Seg}_i) = \sum_{j=1}^{n_i} w(e_{i,j}) \quad (3.4)$$

段预算约束要求每一段的累计权重不超过段预算 B

$$\forall i \in \{0, 1, \dots, m-1\}: W(\text{Seg}_i) \leq B \quad (3.5)$$

定义 3.4（快照承诺） 对第一层快照 $\hat{\sigma}_i$ ，执行方计算状态承诺

$$c_i = \text{Com}(\hat{\sigma}_i) \quad (3.6)$$

其中 Com 为具有计算绑定性的承诺方案（如对状态键值映射的 Merkle 树根）。执行方将承诺序列 $\{c_i\}_{i=0}^m$ 提交至区块链，一经上链不可更改。

一旦计算状态承诺 c_i 上链，执行方无法为同一承诺提供与原始快照不一致的状态数据。这使得承诺序列构成执行方对其快照声明的不可抵赖记录，为后续争议定位提供信任基准。当验证者发现某段 Seg_k 存在异常并确定为争议焦点段后，双方需在该段内进行更细粒度的切分，以精确定位分歧位置。

定义 3.5（裁决阈值） 裁决阈值 b 为系统全局参数，定义链上裁决合约单次重放所能处理的最大指令权重上界。 b 在任务发布时确定，并满足链上预算映射约束：

$$\text{OnChainCost}(b) \leq \Gamma \quad (3.7)$$

其中 Γ 为链上最大可接受裁决开销， $\text{OnChainCost}(\cdot)$ 为将指令权重映射到链上实际执行开销的保守上界函数。裁决阈值 b 与段预算 B 满足 $b \leq B$ ，即裁决粒度不粗于段粒度。

定义 3.6（二次细分动作）对争议焦点段 Seg_k^* ，执行方与挑战者代表各自在链下从 $\hat{\sigma}_k^*$ 出发独立重放该段，按裁决阈值 b 进行细粒度切分，分别生成二次细分状态序列。执行方序列为 $s_0^E, s_1^E, \dots, s_L^E$ ，挑战者序列为 $s_0^C, s_1^C, \dots, s_L^C$ ，满足以下约束：

- (1) 起始状态一致： $s_0^E = s_0^C = \hat{\sigma}_k^*$ 。
- (2) 执行方终止状态与已提交快照承诺一致： $s_L^E = \hat{\sigma}_{k+1}^*$ 。
- (3) 子段预算受限：对任意相邻细分状态间的子段 SubSeg_j ， $W(\text{SubSeg}_j) \leq b$ 。
- (4) 子段数量上界：双方生成长度相同的细分状态序列，子段数记 L ， $L \leq \lfloor B/b \rfloor$ 。

双方分别计算细分状态承诺 $h_j^E = \text{Com}(s_j^E)$ 与 $h_j^C = \text{Com}(s_j^C)$ ，将承诺序列提交至链上。

定义 3.7（争议分歧点）给定执行方与挑战者的二次细分承诺序列 $\{h_j^E\}_{j=0}^L$ 与 $\{h_j^C\}_{j=0}^L$ ，分歧点定义为双方执行轨迹首次出现不一致的位置

$$j^* = \min\{j \mid h_j^E \neq h_j^C\} \quad (3.8)$$

双方提交完整细分承诺且满足边界一致性时，由起始一致性 $h_0^E = h_0^C$ 与终止不一致性 $h_L^E \neq h_L^C$ 可知 j^* 必然存在且 $1 \leq j^* \leq L$ 。在 j^* 处，双方满足 $h_{j^*-1}^E = h_{j^*-1}^C$ （前一位置一致）而 $h_{j^*}^E \neq h_{j^*}^C$ （当前位置不一致），即从同一起始状态出发，双方对该子段的状态转移给出了不同结果。该状态转移即为最小裁决单元。

定义 3.8（最小裁决单元）最小裁决单元（Minimum Dispute Unit, MDU）是包含分歧点 j^* 且满足链上裁决粒度约束的最小执行区间，定义为

$$\text{MDU} = (s_{j^*-1}^E, s_{j^*}^E, s_{j^*}^C) \quad (3.9)$$

其中 $s_{j^*-1}^E$ 为双方一致认可的起始状态， $s_{j^*}^E$ 与 $s_{j^*}^C$ 分别为双方声称的转移结果。MDU的指令权重满足 $W(\text{MDU}) \leq b$ ，结合段预算约束和子段对应执行指令序列，链上进行重放。

定义 3.9（段级验证谓词）段级验证谓词 VerSeg 为链上裁决合约的最终裁决接口。对最小裁决单元 $\text{MDU} = (s_{j^*-1}^E, s_{j^*}^E, s_{j^*}^C)$ ， VerSeg 从起始状态 $s_{j^*-1}^E$ 出发，按确定性语义重放该子段的指令序列直至子段结束，得到重放结果 s' ，判定如下

$$\text{VerSeg}(s_{j-1}^*, s_j^E, s_j^C) = \begin{cases} \text{accept_challenger} & \text{if } s' = s_j^C \\ \text{accept_executor} & \text{if } s' = s_j^E \\ \text{both_wrong} & \text{otherwise} \end{cases} \quad (3.10)$$

VerSeg 是链上最终裁决的核心判定过程。其输入 **MDU**，链上需从 s_{j-1}^* 重放不超过 b 权重的指令并比对结果，即可给出确定裁决。

定义 3.10（任务参数集） 任务发布者在任务发布时确定的系统参数集合定义为

$$\text{Param} = (B, b, \alpha, \Gamma, T_{\text{ch}}, g, T_{\text{stake}}, D(\cdot), \text{deposit_rules}) \quad (3.11)$$

其中 B 为段预算， b 为裁决阈值， α 为密度控制参数， Γ 为链上裁决开销上界， T_{ch} 为挑战窗口期的相关设置， g 为质押阶段最大轮次， T_{stake} 为每轮质押时限， $D(\cdot)$ 为累计质押参数， deposit_rules 为保证金与费用规则。

综上所述，本节将计算任务建模为确定性状态转移系统。执行轨迹通过第一层快照划分为段级区间并以承诺绑定；争议时通过二次细分定位到最小裁决单元；链上裁决通过 **VerSeg** 在预算 Γ 内完成。

3.2.3 裁决正确性模型

本节给出 **CleVer** 的裁决正确性模型，包括协议运行所依赖的前提条件、对手能力刻画以及系统需要满足的正确性性质。

1. 协议安全假设

假设 3.1（诚实验证者存在） 系统中存在至少一个诚实验证者 V^* ，该验证者具备足够的计算资源对争议段进行重放验证，并在发现错误时按协议规定在挑战窗口期 T_{ch} 内发起挑战。该假设是正确性的必要条件，若所有验证者均被腐化或均不作为，则无人发起挑战，错误结果将在窗口期后被默认终局化。

假设 3.2（数据可用性） 诚实验证者能够获得争议段重放所需的必要数据。本章假设挑战窗口期内的相关交互数据均能够正常获得，具体保障机制如链下数据存储与数据可用性层^[37,155]不在本文研究范围之内。本章的裁决正确性分析以该假设为前提。

假设 3.3（确定性执行） 链下计算任务 $\mathcal{T}$ 在同一虚拟机语义下具有完全确定性。给定相同的初始状态 σ_0 与状态转移程序 δ ，执行方和验证方在链下执行或裁决合约在链上重放将产生完全相同的状态转移结果。任务不包含随机化、非确定性调用，仅考虑计算本身。

假设 3.4 (链上可执行性) 裁决合约具备对最小裁决单元执行完整重放的能力。即对任意满足 $W(\text{MDU}) \leq b$ 的最小裁决单元, 裁决合约能够在单次链上调用中完成从起始状态 s_{j^*-1} 出发的指令重放与结果比对。

本章考察经济理性的对手 $\mathcal{A}$ 具备以下能力。

(1) 腐化能力。对手可以腐化执行方 P , 以及验证者集合 $\{V_i\}$ 中除诚实验证者 V^* 以外的任意子集。被腐化的参与方完全受对手控制, 可偏离协议执行任意策略。

(2) 消息调度能力。对手可以观察所有链下通信消息, 并在有限时间范围内调度被腐化方的消息发送顺序与时机。对手无法伪造诚实参与方的消息, 亦无法阻止诚实参与方的链上交易最终被打包上链。

2. 裁决正确性性质

基于上述前提假设与对手模型, 进一步给出 CleVer 需要满足裁决正确性的性质。

(1) 终局唯一性

对同一计算任务, 系统不应终局化两个不同的结果。终局唯一性是其余所有性质的基础前提, 若其被违反, 系统将出现不同参与方看到不同终局结论的情形, 从而破坏验证一致性与经济结算的确定性。

定义 3.11 (终局唯一性) 定义 $\text{Final}(\mathcal{T}, \text{id})$ 表示任务 id 在链上被终局化的结果对象。协议 Π 满足终局唯一性, 若对任意安全参数 λ 与任意 PPT 对手 $\mathcal{A}$

$$\Pr[\exists r_1, r_2 : \text{Final}(\mathcal{T}, \text{id}) = r_1 \wedge \text{Final}(\mathcal{T}, \text{id}) = r_2 \wedge r_1 \neq r_2] \leq \text{negl}(\lambda) \quad (3.12)$$

即对于同一任务, 协议至多接受一个终局结果。

(2) 正确性

若执行方提交了错误的计算结果, 该结果不应被终局化接受。正确性保证系统对错误执行的检测与纠正能力——在诚实验证者存在的条件下, 任何偏离 δ 的执行行为都是可证伪的。

定义 3.12 (正确性) 令 $y^* = \delta^{(N)}(\sigma_0)$ 为正确输出。对手 $\mathcal{A}$ 控制执行方提交候选输出 $\hat{y} \neq y^*$ 及承诺序列。诚实验证者 V^* 在窗口期内按协议行动, 若对任意 PPT 对手, 错误输出被终局化接受且执行方未收到应有惩罚的概率可忽略, 则称协议 Π 满足正确性

$$\Pr[\hat{y} \neq y^* \wedge \text{Final}(\mathcal{T}, \text{id}) = \hat{y} \wedge \text{执行方无惩罚}] \leq \text{negl}(\lambda) \quad (3.13)$$

(3) 完备性

若执行方诚实地按确定性语义 δ 执行并提交正确结果，系统不应错误惩罚执行方，且结果应被终局化。完备性保证诚实参与者不因系统误判而遭受损失。

定义 3.13 (完备性) 诚实执行方按 δ 执行任务 $\mathcal{T}$ ，生成快照与承诺并提交。协议 Π 满足完备性

$$\Pr[\text{Final}(\mathcal{T}, \text{id}) = y^* \wedge \neg \text{Punish}(P)] \geq 1 - \text{negl}(\lambda) \quad (3.14)$$

(4) 活性

任何争议不能被无限拖延。无论对手采取何种策略拖慢验证进度，争议必须在有限时间内达到确定的最终状态。

定义 3.14 (活性/有限终止性) 设协议参数包含挑战窗口期 T_{ch} 、每轮质押时限 T_{stake} 与最大轮次 g 。协议 Π 满足活性，若对任意 PPT 对手 $\mathcal{A}$ 与任意计算任务 id ，一旦进入争议流程，在不超过 T_{resolve} 时间内争议必然终止，其中争议解决时间满足

$$T_{\text{resolve}} = T_{\text{sub}} + g \cdot T_{\text{stake}} + T_{\text{final}} \quad (3.15)$$

其中 T_{sub} 为细分序列提交时限， T_{final} 为链上最终仲裁的执行时间上界。

(5) 链上成本有界

链上成本有界性是 CleVer 将区块链定位为仲裁者而非计算平台的安全保障。无论原始计算任务的规模 N 多大、对手如何构造争议，链上每次最终裁决的执行开销都应受系统参数优先约束。

定义 3.15 (链上成本有界) 协议 Π 满足链上成本有界，若对任意 PPT 对手 $\mathcal{A}$ 与任意争议实例，链上调用 VerSeg 进行最终裁决时的执行开销满足

$$\text{Cost}(\text{VerSeg}(\text{MDU})) \leq \Gamma \quad (3.16)$$

其中 Γ 为链上裁决开销上界，由任务发布时的系统参数预先界定。

3.3 基于有状态任务切片的链下计算验证方案

基于系统模型与安全性质，本节给出 CleVer 的具体方案设计。具体地，自适应状态切片机制在执行阶段将计算轨迹组织为段级区间并持续生成快照，为整个验证体系提供结构化基础。在此基础上，两层切片争议定位机制在争议发生时将分歧精确收敛至最小裁决单元，超线性累计质押协议通过递增的经济压力保障争议在有限轮次内终止。伴随

式验证机制利用切片提供的结构化快照将验证节点前移至执行过程中，降低错误发现延迟与纠错代价。

3.3.1 计算状态快照自适应切片机制

自适应状态切片机制是 CleVer 链下执行的核心组件。其目标是将任意长时间计算的执行轨迹动态划分为满足段预算约束的段级区间，在每个段边界处保存快照并提交承诺，从而为伴随式验证提供实时可用的检查点，并为争议定位提供结构化的索引空间。

定义 3.16 (安全切点谓词) 给定执行过程中时刻 t 的可观测信号 $\text{obs}(t)$ ，具体包括当前执行状态及控制信息，如待执行指令、内存等信息。谓词 $\text{SafeCut}(\text{obs}(t)) \in \{0,1\}$ 判定 t 是否为安全切点。 $\text{SafeCut}(\text{obs}(t)) = 1$ 要求以 t 时刻的状态固定快照后，从该快照出发的后续执行完全确定性且可重放。安全切点谓词的具体实现取决于虚拟机架构。

自适应切片算法在执行中维护当前段的累计权重，通过两类触发条件决定切片时机。

第一类为预算触发，是保证段预算约束的核心机制。算法在执行每条指令前预判，若该指令将导致累计权重超过段预算 B ，则不执行该指令，回退到最近安全切点处结束当前段并固定快照。该机制保证任何段严格满足 $W(\text{Seg}_i) \leq B$ 。预算触发的正确性隐含一个前提，即任意两个相邻安全切点之间的累计权重不超过 B 。在本文所考虑的执行环境中，安全切点通常足够密集，从而该条件在实现上是可满足的。

第二类为提前切片触发，在安全切点处、当前段累计权重达到 αB ($\alpha \in (0,1)$) 时提前结束段。提前切片触发并非可裁决性所必需，而是用于调节切片粒度： α 越小，快照越密，伴随式验证的响应粒度越细，但链下存储开销越大； α 越大，切片越粗，存储负担越轻，但单段验证延迟增大。

执行方在算法运行过程中，对每个生成的快照 $\hat{\sigma}_i$ 计算承诺 $c_i = \text{Com}(\hat{\sigma}_i)$ ，并将承诺序列 $\{c_i\}_{i=0}^m$ 提交至裁决合约。承诺一经上链即不可更改。算法 3.1 给出自适应切片与快照生成的完整过程。核心思想是在执行每一步之前预判是否会超出段预算 B ，若即将超出则在最近安全切点处结束当前段并固定快照。算法运行过程中，执行器在每个安全切点处维护该时刻的状态副本，以支持预算触发时的状态回退。

段预算 B 越大，快照数 m 越少，链下存储与链上承诺提交开销越低，但争议时二次细分产生的子段数 $L \leq \lfloor B/b \rfloor$ 越多，细分承诺序列的链上存储开销越高。 B 越小，快照

更密，伴随式验证粒度更细，但链下存储负担增大。在给定裁决阈值 b 的条件下， B 的选取本质上是链下存储成本与链上争议开销之间的权衡。

算法 3.1：自适应切片算法

输入： 任务 $\mathcal{T}$ ；初始状态 σ_0 ；段预算 B ；权重函数 $w(\cdot)$ ；密度参数 α ；安全切点谓词 SafeCut ；确定性执行器 Exec

输出： 快照序列 $\{\hat{\sigma}_i\}_{i=0}^m$ ；切片边界 $\{t_i\}_{i=0}^m$

- 1: 初始化，令 $(t, i, t_0) \leftarrow 0, \hat{\sigma}_0 \leftarrow \sigma_0$, i 为待写入快照索引
- 2: 段当前累计权重 $W_{\text{cur}} \leftarrow 0$ ，最近安全切点 $t_{\text{safe}} \leftarrow 0$
- 3: 当任务未终止：
- 4: 读取下一条指令 e_{t+1} （仅读取，不执行）
- 5: 计算下一步增量权重 $\Delta w \leftarrow w(e_{t+1})$
- 6: IF $\text{obs}(t)$ 满足安全切点条件
- 7: 更新最近安全切点 $t_{\text{safe}} \leftarrow t$
- 8: IF 提前切片触发 $W_{\text{cur}} \geq \alpha \cdot B$
- 9: $\hat{\sigma}_i \leftarrow$ 固定的当前状态
- 10: 令 $i \leftarrow i+1$ $W_{\text{cur}} \leftarrow 0$
- 11: IF 预算约束触发 $W_{\text{cur}} + \Delta w > B$
- 12: $i \leftarrow i+1$ ，结束当前段
- 13: 回退至 t_{safe} ， $\hat{\sigma}_i \leftarrow$ 固定 t_{safe} 处状态
- 14: 初始化新段累计权重 $W_{\text{cur}} \leftarrow 0$
- 15: 执行 e_{t+1} ， $W_{\text{cur}} \leftarrow W_{\text{cur}} + \Delta w$ ， $t \leftarrow t+1$
- 16: $m \leftarrow i+1$ ， $\hat{\sigma}_m \leftarrow$ 固定终止状态， $t_m \leftarrow t$
- 17: 返回 $\{\hat{\sigma}_i\}_{i=0}^m$ ， $\{t_i\}_{i=0}^m$

3.3.2 基于两层切片的争议定位机制

当验证者在伴随式验证中发现某段重放结果与执行方承诺不一致时，可在挑战窗口期 T_{ch} 内发起挑战。本节给出从挑战发起到分歧点定位的争议定位机制的完整设计方案。该机制的核心设计思想是将争议定位的主要计算工作前移至链下完成，最大程度减少链上开销。记有效挑战所对应的段索引集合为 S 。

1. 挑战收集与焦点段选取

在窗口期 T_{ch} 内，任意验证者均可向链上合约提交挑战交易，声明其发现异常的段索引 k 并缴纳初始保证金。链上合约对每笔挑战进行有效性检查，如索引是否在 $\{0, \dots, m-1\}$ 范围内、保证金是否充足等，通过后将索引加入被挑战段集合 S 。

窗口期结束后,若 $S = \emptyset$ 则无有效挑战,任务结果被确认,流程进入阶段五。若 $S \neq \emptyset$,系统按以下规则处理。系统按最小索引优先原则选取争议焦点段

$$k^* = \min(S) \quad (3.17)$$

由于执行轨迹具有前缀依赖关系,较早段中的错误会沿后续状态传播。若段 k^* 存在错误,则 σ_{k^*+1} 及其后续所有快照均建立在错误状态之上,后续段的争议均为该错误的派生结果。因此,优先处理最早争议段可避免对派生错误的重复裁决,对 k^* 的单次裁决足以对任务整体给出结论。

若多个验证者对段 k^* 提出挑战,选取初始保证金最高者为挑战者代表,由其在后续流程中与执行方对抗,其余有效挑战者的初始保证金不再单独参与后续对抗,而是统一记入挑战方公共质押池,仅用于代表挑战者后续轮次中的追加质押。保证金相同则取最先提交者。对于未被选为代表但先前提交有效挑战的其他验证者,其初始保证金不被罚没,并可按预先约定规则分享成功挑战收益,从而维持系统对诚实监督行为的持续激励。

2. 二次细分序列提交

焦点段 k^* 确定后,执行方与挑战者代表需各自对该段进行二次细分并提交承诺序列。执行方首先在时限 T_{sub} 内提交其细分承诺序列 $\{h_j^E = \text{Com}(s_j^E)\}_{j=0}^L$ 。裁决合约校验起始一致性 $h_0^E = c_{k^*}$ 与终止一致性 $h_L^E = c_{k^*+1}$, 确保执行方的细分序列与已上链的快照承诺相同。

随后挑战者代表在时限内提交其细分承诺序列 $\{h_j^C = \text{Com}(s_j^C)\}_{j=0}^L$ 。裁决合约校验起始一致性 $h_0^C = c_{k^*}$ 与终止不一致性 $h_L^C \neq h_L^E$, 若挑战者终止状态与执行方一致,则无实质争议,挑战被驳回。若任一方未按时提交或校验失败,该方直接判负,争议终止。双方承诺序列一经上链即不可更改,作为后续争议解决的不可抵赖依据。

双方细分承诺序列上链后,需定位分歧点 $j^* = \min\{j \mid h_j^E \neq h_j^C\}$ 。由起始一致性 $h_0^E = h_0^C$ 与终止不一致性 $h_L^E \neq h_L^C$ 可知 j^* 必然存在。

分歧点的定位由双方在链下对比承诺序列完成,这一过程仅需 $O(L)$ 次哈希值比较,计算量可忽略。若双方就 j^* 达成一致,任一方向裁决合约提交 j^* ; 若双方无法达成一致,以累计质押较高方所提出的争议点为准。无论通过何种方式确定 j^* , 裁决合约仅需验证以下条件即可接受该分歧点声明

$$h_{j^*-1}^E = h_{j^*-1}^C \quad \wedge \quad h_{j^*}^E \neq h_{j^*}^C \quad (3.18)$$

至此，争议被收敛至最小裁决单元 $\text{MDU} = (s_{j^*-1}^E, s_{j^*}^E, s_{j^*}^C)$ 。整个定位过程中，链上仅承担挑战交易收集、承诺序列存储与分歧点条件校验，争议的主要收敛工作均在链下完成。图 6 展示了 CleVer 在争议发生后，从挑战收集、焦点段选取、二次细分与分歧点定位，到争议收敛至最小裁决单元并进入超线性累计质押阶段的完整衔接过程。

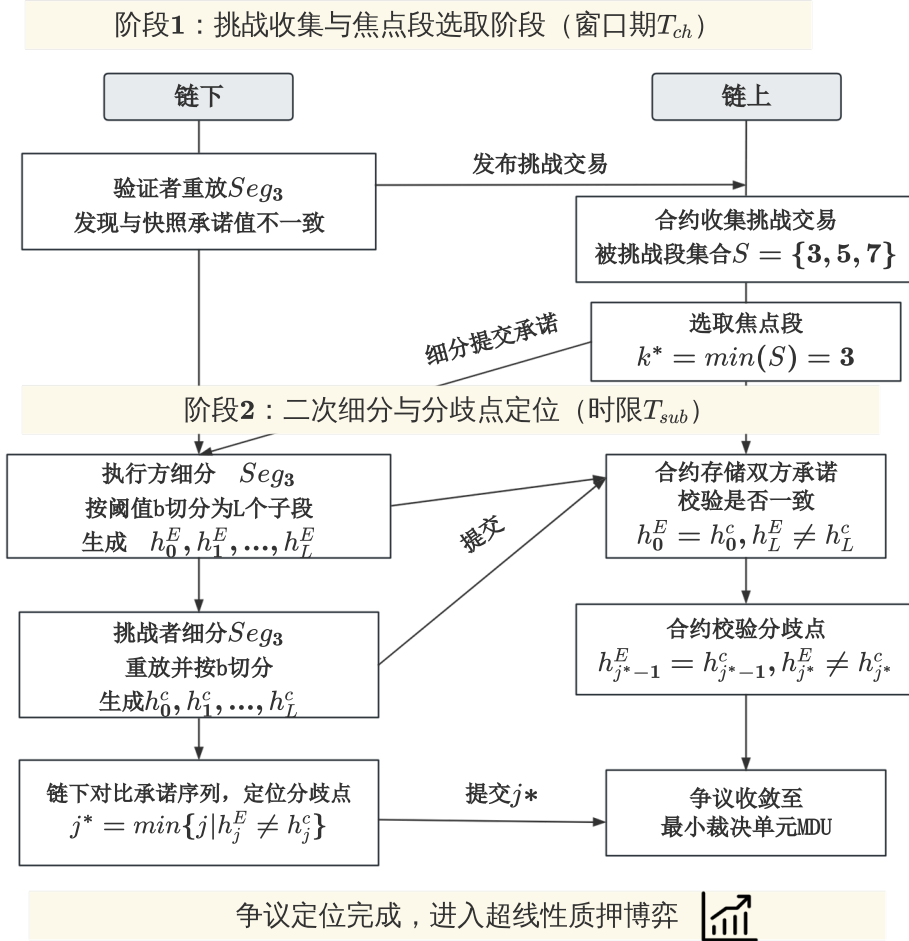


图 6 CleVer 两层定位流程图

3.3.3 超线性累计质押的争议解决协议

经上一节的争议定位，焦点段 k^* 已确定且双方细分承诺序列已上链。本节给出从细分序列提交到最终裁决的完整争议解决协议。

协议的核心思想是，在进入链上最终裁决之前，通过超线性累计质押博弈对争议双方施加持续递增的经济压力，使错误方在理性条件下倾向于尽早退出。只有双方均坚持到底时，才触发链上重放裁决。需要指出的是，超线性质押机制仅改变争议的终止路径，使多数争议通过经济博弈提前终止，但并不改变最终裁决的正确性标准。若双方坚持到最后，裁决仍由 VerSeg 基于确定性重放给出。

1. 超线性累计质押规则

定义 3.17 (超线性累计质押曲线) 累计质押曲线 $D: \{1, \dots, g\} \rightarrow \mathbb{R}^+$ 满足下述三种特性:

- (1) 初始正值, 即 $D(1) > 0$ 。
- (2) 严格递增, 即 $D(r+1) > D(r)$, $\forall r \in \{1, \dots, g-1\}$ 。
- (3) 增量超线性, 即 $\Delta D(r) = D(r) - D(r-1)$ 关于 r 严格递增, $\forall r \geq 1$ 。

增量超线性曲线下, 参与方如果决定继续坚持, 每多坚持一轮需追加的质押比上一轮更多, 使继续参与的边际成本持续增长。典型增量曲线包括二次型 $D(r) = d_0 \cdot r^2$ 与指数型 $D(r) = d_0 \cdot \beta^r$, 其中 d_0 为基础质押系数。

双方细分承诺序列上链后, 进入至多 g 轮的累计质押阶段。在第 r 轮, 双方各自需在时限 T_{stake} 内将累计质押补足至 $D(r)$ 。若任一方未按时补足, 如主动放弃或超时, 该方视为退出并直接判负, 争议直接终止。

2. 经济激励分析

超线性质押机制的经济激励逻辑建立在以下观察之上。

在争议信息相对透明且外部资金方能够形成胜败概率判断的条件下, 由于确定性执行假设, 正确方知道自己在链上最终裁决中必胜, 因此坚持到底的预期收益为正; 而错误方知道自己必败, 继续质押会增加最终损失。由于 $D(\cdot)$ 严格递增, 错误方在任意轮次退出的损失均严格小于继续坚持的损失。

定理 3.1 (错误方的严格退出均衡) 在完全信息且双方能够准确预见最终裁决结果的条件下, 错误方在任意轮次继续质押均不会改善其收益, 因此其最优策略倾向于尽早退出。

证明 考察任意 $r \in \{1, \dots, g\}$, 错误方在轮次 r 面临两种选择。若退出, 损失为 $D(r-1)$ ($r=1$ 时为 0)。若继续补足至 $D(r)$, 后续要么在某轮 $r' > r$ 退出, 损失为 $D(r'-1)$; 要么坚持到最终裁决必败, 损失为 $D(g)$ 。由 $D(\cdot)$ 严格递增, 继续后的最小可达损失为 $D(r)$ 。

由 $D(r) > D(r-1)$ 知退出的损失 $D(r-1)$ 严格小于继续的最小损失 $D(r)$, 退出严格优于继续。因而在博弈模型下, 错误方选择尽早退出构成自然的子博弈精炼均衡。 $\square$

进一步考虑不完全信息场景。设某方对自己败诉的主观概率为 p 。其坚持到最终裁决的期望收益为

$$\mathbb{E}[\text{坚持}] = (1-p) \cdot D(g) - p \cdot D(g) = (1-2p) \cdot D(g) \quad (3.19)$$

推论 3.1（不完全信息下的退出条件） 当 $p > 0.5$ 时，坚持到最终裁决的期望收益为负。由于 $D(\cdot)$ 的超线性增长使 $D(g)$ 远大于前期质押，预期损失被显著放大，越早退出越有利。更一般地，对于在轮次 r 面临继续或退出决策的参与方，当其败诉信念大于 $D(r-1)/(D(r-1)+D(g))$ 时，退出为理性选择。

在 CleVer 的协议设计中，争议双方的质押不限于自有资金，允许从外部获得资金支持。在信息透明的条件下，若外部资金方能够评估争议双方的胜败概率，则理性的外部资金方倾向于资助胜率更高的一方以获取分成收益。这一机制在正确方资金不足时可提供缓解途径，但其实际效果取决于信息透明度与外部资金市场的效率，本文不将其作为协议正确性的依赖条件。

3. 仲裁路径

质押阶段结束后，仲裁由两条路径触发。

(1) 路径一：提前退出。若在某轮 $r^* < g$ 中一方退出，争议立即终止。退出方判负，未退出方判胜。胜者收回自身质押并获得对方全部累计质押 $D(r^*-1)$ ，败者损失全部累计质押。此路径不触发链上重放，争议完全通过经济博弈解决。

(2) 路径二：链上最终裁决。若双方完成全部 g 轮均未退出，进入链上最终裁决，该路径分三步执行。

①分歧点提交。双方在链下对比各自的细分承诺序列确定分歧点 j^* 。任一方向裁决合约提交 j^* ，合约验证是否成立。

②状态数据提交与验证。双方在规定时间内向裁决合约提交分歧点处的实际状态数据：起始状态 s_{j^*-1} 、执行方声称的 $s_{j^*}^E$ 与挑战者声称的 $s_{j^*}^C$ ，并附上各自数据与此前上链承诺的一致性证。裁决合约验证 $\text{Com}(s_{j^*-1}) = h_{j^*-1}^E, \text{Com}(s_{j^*}^E) = h_{j^*}^E, \text{Com}(s_{j^*}^C) = h_{j^*}^C$ ，若某方数据与承诺不一致或未按时提交，该方直接判负。

③链上重放裁决。裁决合约加载起始状态 s_{j^*-1} ，按确定性语义 δ 重放该子段指令直至子段结束，得到重放结果 s' 。裁决结果的后续处理分为三种情形：

挑战者胜出，执行方在该子段的状态转移中出错。执行方判负，没收执行方全部累计质押转移给挑战者。 $\hat{\sigma}_k$ 被标记为最后正确快照，系统从该快照处安排重新执行。

执行方胜出，挑战者声称的转移结果有误。挑战者判负，没收挑战者全部累计质押转移给执行方。任务结果被终局化。

双方声称的结果均与链上重放结果不一致。没收双方全部累计质押。 $\hat{\sigma}_k^*$ 被标记为最后正确快照，系统从该快照处安排重新执行。

定义 3.18 (争议收益) 争议结束时双方累计质押均为 $D(r^*)$ ， r^* 为结束轮次，质押池总额 $P(r^*) = 2D(r^*)$ 。胜者的收益为 $+D(r^*)$ ，并且可以收回自身质押并赢得对方全部质押。而败者净收益 $-D(r^* - 1)$ ，意味着其会损失全部累计质押。当双方都错误时，双方各损失 $D(r^*)$ ，质押池由系统收回。

3.3.4 基于伴随式验证的错误回退机制

伴随式验证的核心特征是，将验证时间点由任务结束后前移到执行过程中，从而将纠错代价从全局回滚降为局部的段级恢复。由于自适应切片机制在执行过程中持续生成快照并提交承诺，验证者无需等待计算完成即可对已有段展开验证。伴随式验证的具体工作流程如下。

当裁决合约上出现新的快照承诺 c_{i+1} 后，验证者从链下获取快照 $\hat{\sigma}_i$ 与段 Seg_i 的执行数据，在本地从 $\hat{\sigma}_i$ 出发按确定性语义 δ 重放至段结束，将重放结果与 c_{i+1} 比对。若结果一致，验证者继续等待下一个快照承诺；若结果不一致，验证者在挑战窗口期 T_{ch} 内向裁决合约提交挑战交易。验证者的验证进度与执行方的执行进度形成流水线关系：执行方在生成段 Seg_{i+1} 的同时，验证者可并行验证段 Seg_i 。验证者的验证延迟主要取决于单段的重放时间与快照数据的获取时间，而非整个任务的执行时间。

在伴随式验证架构下，错误被发现的时间点接近该错误段的快照生成时间，而非任务的全部执行结束时间。具体而言，若执行方在段 Seg_k 中引入了错误的状态转移，验证者在快照 $\hat{\sigma}_{k+1}$ 的承诺上链后即可通过重放发现异常，错误发现延迟约为单段执行时间加上快照数据获取时间。

一旦验证者确认异常并发起挑战，争议处理流程立即启动，而无需等待后续段的执行完成。这意味着在长时间任务的执行过程中，错误可以被及时截断，避免系统在已确认异常的基础上继续长时间推进无效计算。

当争议裁决认定执行方在段 Seg_k^* 中出错后，系统将 $\hat{\sigma}_k^*$ 标记为最后正确快照，后续计算从 $\hat{\sigma}_k^*$ 处接续执行。由此，计算资源的浪费被严格限制在一个段级区间内——仅段 Seg_k^* 内错误点之后的计算需要重做，而非从任务起始状态重新执行全部计算。

此外, 由于验证负载在时间维度上被分摊到各段的验证过程中, 到任务执行完成时, 验证者已覆盖大部分中间状态。

3.4 理论分析

本节围绕系统性质与裁决正确性要求, 按证明依赖关系给出形式化分析。首先分析可裁决性, 证明两层切片机制保证争议能够收敛至链上可执行的最小裁决单元且裁决开销受控; 在此基础上分析裁决正确性, 证明终局唯一性、正确性与完备性; 最后分析有限终止性, 证明争议在有限时间内必然结束。本节记 CleVer 计算验证协议为 Π 。

3.4.1 可裁决性分析

可裁决性是后续理论分析的基础, 若不能保证争议收敛至链上可执行的最小裁决单元, 裁决正确性和有限终止性均无法成立。本小节依次证明段预算约束、子段预算约束与链上成本有界性, 最后给出切片规模的辅助估计。

引理 3.1 (段预算不越界) 在自适应切片算法 3.1 中, 对执行时切片生成的段 Seg_i , 均有 $W(\text{Seg}_i) \leq B$ 。

证明 证明以下性质在算法执行过程中始终成立, 在段内任意时刻, 已执行步的累计权重 W_{cur} 不超过 B 。

段开始时 $W_{\text{cur}} = 0$, 不变式自然成立。段执行过程中, 算法在执行每条指令 e_{t+1} 前检查 $W_{\text{cur}} + w(e_{t+1}) > B$ 是否成立。若成立即预算触发, 算法回退至最近安全切点 t_{safe} 处结束当前段, 此时段内累计权重为截至 t_{safe} 的累计值, 不超过触发前的 $W_{\text{cur}} < B$ 。若不成立, 执行该指令后 $W_{\text{cur}} \leftarrow W_{\text{cur}} + w(e_{t+1}) \leq B$, 不变式仍成立。提前切片触发在 $W_{\text{cur}} \geq \alpha B$ 时结束段, 由 $\alpha \leq 1$ 知 $W_{\text{cur}} \leq B$ 。

段结束时的累计权重即为 $W(\text{Seg}_i)$, 由不变式知 $W(\text{Seg}_i) \leq B$ 。 $\square$

引理 3.1 保证了第一层切片的预算约束严格成立, 为二次细分提供了前提, 只有段级权重被控制在 B 以内, 按裁决阈值 b 的再切分才能产生数量有限的子段。

引理 3.2 (二次细分子段预算不越界) 对争议焦点段 Seg_k 的二次细分, 每个子段满足 $W(\text{SubSeg}_j) \leq b$, $\forall j \in \{0, \dots, L-1\}$ 。

证明 二次细分按裁决阈值 b 对段内执行进行切分，其切分逻辑与自适应切片算法的预算触发机制相同（以 b 替代 B ）。由引理 3.1 的证明方法直接可得每个子段的累计权重不超过 b 。□

两层预算约束联合保证了争议最终能收敛到权重不超过 b 的最小裁决单元。同时下述定理 3.2 建立了该结论与链上裁决开销上界 Γ 之间的关联。

定理 3.2（链上成本有界性） 在计算任务确定性执行与链上可执行的假设前提下，协议 Π 满足链上成本有界性。

证明 链上最终裁决调用 $\text{VerSeg}(s_{j-1}^E, s_j^E, s_j^C)$ ，其执行过程为，从 s_{j-1}^E 出发按 δ 重放至子段结束，并将结果与 s_j^E 、 s_j^C 比对。由定义 3.8 最小裁决单元可知，MDU 对应的子段即为 SubSeg_{j-1} ，该子段的指令权重满足 $W(\text{MDU}) = W(\text{SubSeg}_{j-1}) \leq b$ 。由系统参数约束 $\text{OnChainCost}(b) \leq \Gamma$ ，链上重放开销不超过 Γ 。比对操作的开销为常数级，不影响上界。因此 $\text{Cost}(\text{VerSeg}(\text{MDU})) \leq \Gamma$ 。□

推论 3.2（切片规模上界） 设任务总累计权重为 $W_{\text{total}} = \sum_{i=1}^N w(e_i)$ ，并假设单条指令的最大权重 w_{max} 满足 $w_{\text{max}} \leq (1-\alpha)B$ ⁴。再给定切点密度与权重约束下，段数 m 上界为

$$m \leq \left\lceil \frac{W_{\text{total}}}{\alpha \cdot B} \right\rceil \quad (3.20)$$

证明 考察非最后一段。段结束由预算触发或提前切片触发引起。对于预算触发，触发条件为 $W_{\text{cur}} + w(e_{t+1}) > B$ ，此时算法回退至最近安全切点 t_{safe} 处结束段。由 $w_{\text{max}} \leq (1-\alpha)B$ ，触发时有 $W_{\text{cur}} > B - w_{\text{max}} \geq B - (1-\alpha)B = \alpha B$ 。对于提前切片触发，触发条件为 $W_{\text{cur}} \geq \alpha B$ ，段内累计权重直接满足不小于 αB 。两种情况下，非最后一段的累计权重均不小于 αB 。总权重 W_{total} 均分到 m 个段中，其中最后一段可能不足 αB ，故 $m \leq \lceil W_{\text{total}} / (\alpha \cdot B) \rceil$ 。□

3.4.2 裁决正确性分析

定理 3.3（终局唯一性） 在区块链一致性与终局性成立前提下，协议 Π 满足终局唯一性。

证明 CleVer 中任务结果的终局化通过链上合约状态转移实现：要么在窗口期无挑战后由合约将结果标记为已确认，要么在争议裁决后由合约依据 VerSeg 的输出确定结果。两种路径均为链上交易触发的合约状态变更。基于一致性和终局性假设，所有诚实节点

⁴ 在以太坊 EVM 中，单条操作码的最大 Gas 消耗（如 CREATE 类指令）约为 32000 Gas。在默认参数配置下， $(1-\alpha)B$ 远大于 w_{max} ，因此该条件在实际执行环境中自然满足。

对合约状态达成一致，并且状态变更一旦终局化不可回滚。因此，对同一任务标识符 id ，合约至多产生一个终局化结果。产生两个不同终局化结果的概率为 $\text{negl}(\lambda)$ 。 $\square$

定理 3.4（理性对手下的正确性） 在诚实验证者存在、数据可用性、确定性执行以及经济理性对手假设下，协议 Π 满足正确性，即错误执行结果不会被系统接受。

正确性证明的核心在于建立从“错误结果被提交”到“错误结果被否决”的完整推导过程。该推导依赖两个关键环节：一是诚实验证者必然发现错误并发起挑战，二是焦点段选取规则保证争议聚焦在有效的错误段上。

证明 将该定理的证明转化为证明腐化执行方提交了错误结果 $\hat{y} \neq y^*$ 。需证明在诚实验证者 V^* 按协议行动的条件下，该错误结果不会被终局化接受。证明分为以下三个步骤：

（1）步骤一：错误段的存在性与诚实挑战。由 $\hat{y} \neq y^*$ 与执行轨迹的串联依赖性，必存在第一个错误段 $k_0 = \min\{i \mid \hat{\sigma}_{i+1} \neq \delta(\hat{\sigma}_i)\}$ 。诚实验证者 V^* 对执行轨迹进行伴随式验证；根据数据可用性假设， V^* 能够获得段 Seg_{k_0} 的重放数据。 V^* 在验证该段时将发现重放结果与承诺 c_{k_0+1} 不一致，并在窗口期 T_{ch} 内发起挑战，将 k_0 纳入被挑战段集合 S 。

（2）步骤二：焦点段的有效性。由焦点段选取规则， $k^* = \min(S)$ 。由于诚实验证者必然对第一个真实错误段 k_0 发起挑战， $k_0 \in S$ 。在理性对手假设下，即腐化验证者不愿承担必败经济损失，其不会对 k_0 之前的正确段发起虚假挑战，因为虚假挑战必然失败，将导致攻击者损失全部累计质押。因此 $k^* = k_0$ ，焦点段即为第一个真实错误段。

（3）步骤三：裁决结果。对焦点段 k^* 的裁决分争议双方有一方退出或仍然存在争议两种情形。若执行方在质押阶段某轮退出。由协议规则，退出方判负，执行方累计质押被没收，任务从 $\hat{\sigma}_{k^*}$ 处重新执行。错误结果未被终局化；若双方坚持 g 轮，触发链上最终裁决。裁决合约加载 s_{j^*-1} 并重放至子段结束，得到唯一确定的结果 s' ，且 s' 为从 s_{j^*-1} 出发的唯一正确转移结果。此时链上裁决有如下结果：

①若 $s' = s_{j^*}^C \neq s_{j^*}^E$ ：裁决为 `accept_challenger`，执行方判负，从 $\hat{\sigma}_{k^*}$ 重新执行。

②若 $s' \neq s_{j^*}^E$ 且 $s' \neq s_{j^*}^C$ ：裁决为 `both_wrong`，双方质押均被没收，从 $\hat{\sigma}_{k^*}$ 重新执行。

③裁决为 `accept_executor` 的情形在执行方确实错误的前提下不会发生。由确定性语义，若执行方在段 k^* 上的执行结果错误，则其细分序列中必存在某个子段转移与正确结果不一致。该子段即为分歧点 j^* 所在的子段。链上从 s_{j^*-1} 出发的重放结果 s' 为唯一正确结果，不可能与执行方的错误声称 $s_{j^*}^E$ 一致。

综上，在诚实验证者按协议行动的条件下，若执行方提交错误结果，则协议要么在争议阶段直接判定执行方败诉并施加惩罚，要么通过链上最终裁决否决该错误结果并从最后正确快照处恢复执行。因此，错误输出被终局化接受且执行方未受到应有惩罚的发生概率可忽略，故协议满足正确性。 $\square$

若腐化验证者对 k_0 之前的正确段发起虚假挑战，可能使 $k^* < k_0$ ，导致争议聚焦在正确段上。在该情形下，执行方将在链上裁决中胜出，挑战者损失全部累计质押，任务继续执行。但真正的错误段 k_0 在此轮争议中未被处理。在理性对手假设下，虚假挑战败诉将导致攻击者损失全部累计质押 $D(g)$ ，理性对手不会为偏移焦点段而承担此经济损失，因此该攻击不成立。

裁决的正确性保证了错误结果不会被接受。与之相对的，下述完备性定理保证正确行为不会被惩罚。

定理 3.5（完备性） 在确定性执行假设下，若裁决合约按协议执行且链上重放语义与链下执行语义一致，则协议 Π 满足完备性，即诚实参与者不会因系统误判而遭受损失。

证明 设诚实执行方按 δ 执行任务 $\mathcal{T}$ ，并按自适应切片生成快照序列 $\{\hat{\sigma}_i\}_{i=0}^m$ 与承诺序列 $\{c_i\}_{i=0}^m$ 。由确定性执行假设，执行方的快照序列忠实反映确定性执行轨迹，且承诺序列与快照相对应。证明分两种情形考察：

(1) 无挑战情形。窗口期 T_{ch} 内无验证者发起挑战，任务结果被终局化。诚实执行方的输出等于 y^* ，无惩罚事件发生。完备性成立。

(2) 遭遇挑战情形。假设某验证者对段 k^* 发起挑战。诚实执行方按协议提交段 Seg_{k^*} 的细分承诺序列。此时，若挑战者在质押阶段退出，执行方胜出，无惩罚。若进入链上最终裁决，裁决合约从 s_{j^*-1} 出发按确定性语义 δ 重放得到结果 s' 。由于链下执行与链上重放语义一致性，诚实执行方的转移结果 s_j^E 与链上重放结果 s' 一致，二者均从同一起始状态 s_{j^*-1} 出发，按相同的确定性语义 δ 执行，且对手无法伪造链上重放所依赖的数据绑定关系。此时裁决结果为 `accept_executor`，执行方胜出。

两种情形下诚实执行方均不被惩罚，且正确结果被终局化。 $\square$

3.4.3 有限终止性分析

有限终止性要求争议不能被无限拖延。以下定理证明争议解决时间存在确定性上界。

定理 3.6（争议解决有限终止性） 协议 Π 满足有限终止性，其争议解决时间不超过

$$T_{\text{resolve}} = T_{\text{sub}} + g \cdot T_{\text{stake}} + T_{\text{final}}。$$

证明 争议流程由以下阶段串联组成，每个阶段均有严格的时间上界。

(1) 细分序列提交阶段。双方在 T_{sub} 内提交细分承诺序列。若某方超时，该方直接判负，争议终止。该阶段耗时至多 T_{sub} 。

(2) 质押阶段。至多 g 轮，每轮时限 T_{stake} 。若某轮某方超时或未补足，该方判负，争议终止。该阶段耗时至多 $g \cdot T_{\text{stake}}$ 。

(3) 链上最终裁决阶段。当双方完成 g 轮后，分歧点提交、数据验证与链上重放均在有限步内完成。由定理 3.2，重放开销不超过 Γ ，可在有限时间内执行完毕。若某方未按时提交数据，直接判负。该阶段耗时至多 T_{final} 。

所有时限以链上区块高度定义，链下网络延迟只会影响参与方能否及时响应，而不会改变协议给出的最坏时间上界。三个阶段串联，总耗时不超过 $T_{\text{resolve}} = T_{\text{sub}} + g \cdot T_{\text{stake}} + T_{\text{final}}$ 。每个阶段要么推进到下一阶段，要么因超时直接终止争议，不存在无限等待的路径。 $\square$

综上所述，本节围绕裁决正确性、链上成本有界性与有限终止性三个维度完成了 CleVer 协议的理论分析。可裁决性分析证明了两层切片机机制保证争议收敛至链上可执行的最小裁决单元，且裁决开销受系统参数 Γ 约束；裁决正确性分析证明了在诚实验证者存在、数据可用、确定性执行与理性对手的前提下，错误结果不被终局化接受且诚实参与者不被误惩罚；有限终止性分析证明了争议在确定性时间上界 T_{resolve} 内终止。上述分析为 CleVer 作为链下计算验证方案提供了围绕裁决正确性、链上成本控制与协议终止性的理论保障。

3.5 实验分析

本节通过实验对 CleVer 系统的性能与资源开销进行系统分析。实验围绕本章所提出的核心机制展开，主要包括自适应切片机机制的切片开销、争议处理效率以及伴随式验证的时效性三个方面。

3.5.1 实验设置

链下执行环境采用基于 Geth 的以太坊虚拟机 EVM，通过在 EVM 执行循环中注入切片检查逻辑，在自适应切片算法判定的段边界处，调用 StateDB 的快照接口将当前账

户状态、合约内存及程序计数器序列化为可恢复的状态检查点文件，从而支持运行时状态快照的保存与从任意快照处的确定性重放加载。链上裁决合约以 Solidity 编写并部署于本地以太坊测试网络，用于模拟链上裁决执行与 Gas 开销测量。硬件配置按照以太坊全节点推荐规格：Intel i7-12700，12 核 24 线程 4.9GHz，内存为 32GB DDR4，硬盘存储为 2TB NVMe SSD。

本文以 EVM 的 Gas 消耗量作为计算规模的统一度量。Gas 消耗量能够较为直接地刻画任务在链上重放时的执行开销，与段预算 B 、裁决阈值 b 等系统参数的含义一致。为全面评估 CleVer 在不同计算下的表现，选取四类基准任务，具体描述如表 3 所示。

表 3 CleVer 测试基准任务描述

具体计算任务	计算任务描述	执行消耗量	执行时间量级
Fibonacci	迭代计算第 n 个 Fibonacci 数	10^9	十秒级
Poly-Chain	对连续执行 n 轮多项式递推计算	10^{10}	分钟级
Sort-Large	对大规模数组执行归并排序	10^{11}	十分钟级
DP-Large	对大规模输入执行动态规划求解	10^{12}	小时级

四类任务的设计考量如下。Fibonacci 任务以大数迭代为核心，计算逻辑简单但涉及频繁的大数算术运算，状态空间较小，适合作为基础基准评估切片机制的基本开销。Poly-Chain 任务在每轮迭代中执行多项式求值与状态更新，状态复杂度中等，适合评估切片机制在连续状态演化场景下的表现。Sort-Large 任务对大规模数组执行归并排序，涉及密集的内存读写与比较操作，状态空间较大，适合评估快照序列化在高内存占用场景下的开销特征。DP-Large 任务执行大规模动态规划求解，执行时间最长、状态持续演化，代表了 CleVer 所面向的验证长时间执行的复杂计算任务场景。所有任务均在确定性执行环境中运行。

除特别说明外，实验采用以下默认参数配置。段预算 $B = 10^8$ Gas，裁决阈值 $b = 10^6$ Gas，密度控制参数 $\alpha = 0.8$ ，质押阶段最大轮次 $g = 10$ ，累计质押曲线采用二次型 $D(r) = d_0 \cdot r^2$ ， $d_0 = 0.5$ ETH。

本节选取以下四种链下计算验证的主要技术路线作为对比基准，涵盖了当前主流的争议解决机制设计。各方案均采用其原生执行环境以反映实际系统特性。

(1) Arbitrum 方案^[71]。采用标准 1v1⁵二分协议，执行方与挑战者通过链上交互逐步二分争议范围至单步指令，交互轮次为 $O(\log N)$ ，其中 N 为计算步数。验证者需获得许可，争议按顺序处理。

(2) TrueBit 方案^[69]。引入求解者与验证者的角色分离机制，以及基于强制错误注入的奖金激励机制；争议定位仍采用 1v1 二分方式进行，交互轮次为 $O(\log N)$ ，保证金在争议期间保持恒定。

(3) Cartesi 方案^[156]。基于无许可裁判锦标赛算法，将多方争议组织为淘汰赛结构，每场比赛内部使用二分协议。单次争议的交互轮次为 $O(\log N)$ ，多对手场景下额外增加锦标赛调度开销，最终裁决基于有效性证明。

(4) BoLD 方案^[74]。采用三级二分协议，支持全许可 All-vs-All 争议⁶。三级累计交互轮次仍为 $O(\log N)$ 量级，具备延迟攻击抵抗能力。

表 4 给出五种方案的综合对比。

表 4 CleVer 与典型方案对比分析

对比维度	Arbitrum ^[71]	TrueBit ^[69]	Cartesi ^[156]	BoLD ^[74]	CleVer
争议机制	二分协议	二分协议	锦标赛结构	三级二分协议	两层切片+质押
链上交互轮次	$O(\log N)$	$O(\log N)$	$O(\log N)$	$O(\log N)$	常数轮次
争议延迟上界	无上界	无上界	$O(\log N_d)$ 周期	2 个挑战周期	固定参数上界
多方争议支持	顺序 1v1	求解者-验证者	锦标赛淘汰	All vs All 协作	最小索引+代表
验证模式	事后验证	事后验证	事后验证	事后验证	伴随式验证
链上最终裁决	虚拟机单步证明	虚拟机单步证明	局部证明单元	虚拟机单步证明	最小裁决单元
限制争议拖延	否	否	否	是	是

3.5.2 自适应切片机制分析

本节评估自适应切片机制的切片质量、执行开销与参数敏感性。对四类基准任务，分别在 $B \in \{10^6, 10^7, 10^8, 10^9\}$ 下运行自适应切片算法，记录每段的实际累计 Gas 权重。

图 7 以箱线图展示了段权重占预算百分比 $W(\text{Seg}_i)/B$ 的分布。

⁵ 1v1 表示一对一争议模式，即每次争议仅在单个挑战者与单个被挑战方之间展开。

⁶ All vs All 表示多方同时参与的争议模式，即所有具备资格的参与方均可对同一结果发起异议并进入裁决。

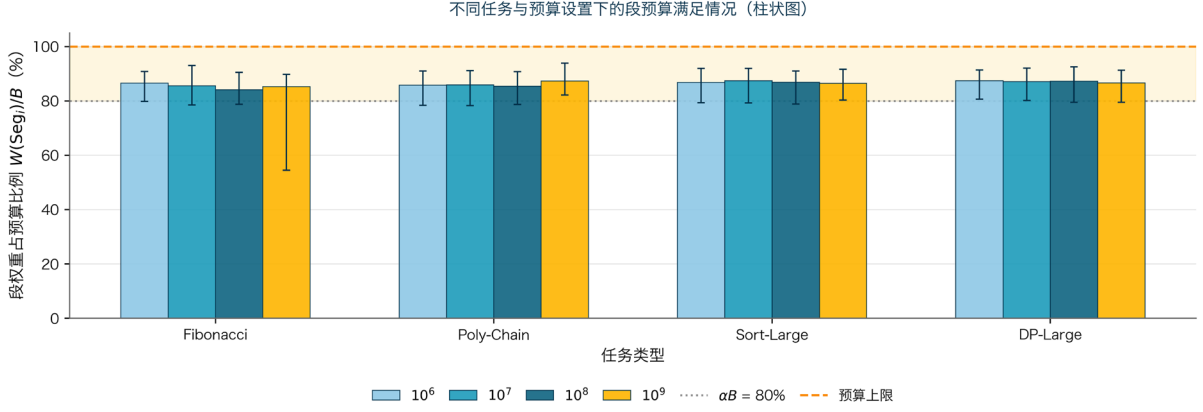


图 7 段权重占预算百分比

实验结果表明，所选取的测试任务其段权重均不超过 B ，验证了引理 3.1 段预算不越界性质在实际系统中的可实现性。多数段的权重集中在 αB 附近，这表明段预算被充分利用，并且提前切片触发机制能够在控制切片粒度的同时避免明显的过度碎片化。当 B 接近任务总 Gas 量时，末段占比降低，但预算约束仍严格成立。

随后对四类任务分别测量无切片的原始执行时间与启用切片的执行时间，计算切片开销比并分解开销构成。图 8 展示了实验测试结果。

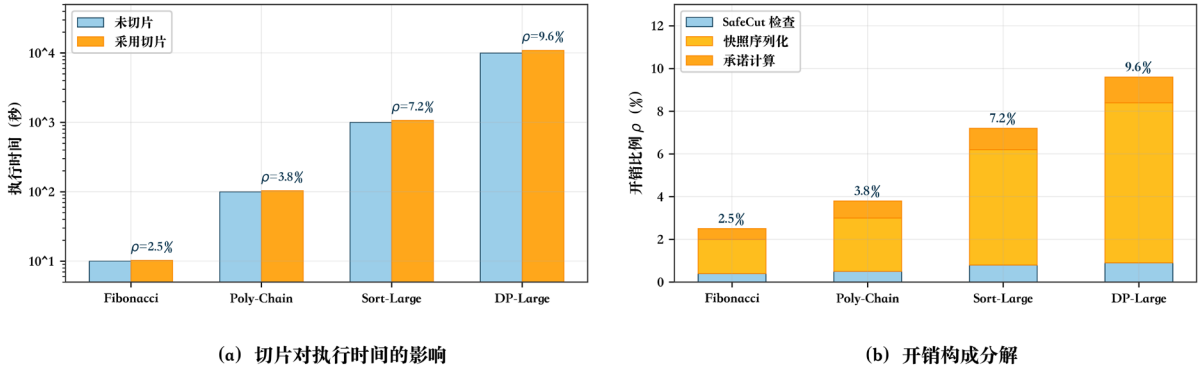


图 8 任务切片执行开销分析

四类任务的切片开销比 ρ 均不超过 10%。从开销构成来看，快照序列化是主要开销来源，其占比随任务状态空间的增大而增长。安全切点检查开销在所有任务中保持稳定，因其仅涉及轻量级的执行状态谓词判定。承诺计算开销占比最小且增长平缓。上述结果表明，自适应切片机制为执行轨迹引入的结构化验证能力，其额外执行代价是可控的。

段预算 B 与裁决阈值 b 是 CleVer 的两个核心参数。图 9 分别展示了二者对系统开销的影响。

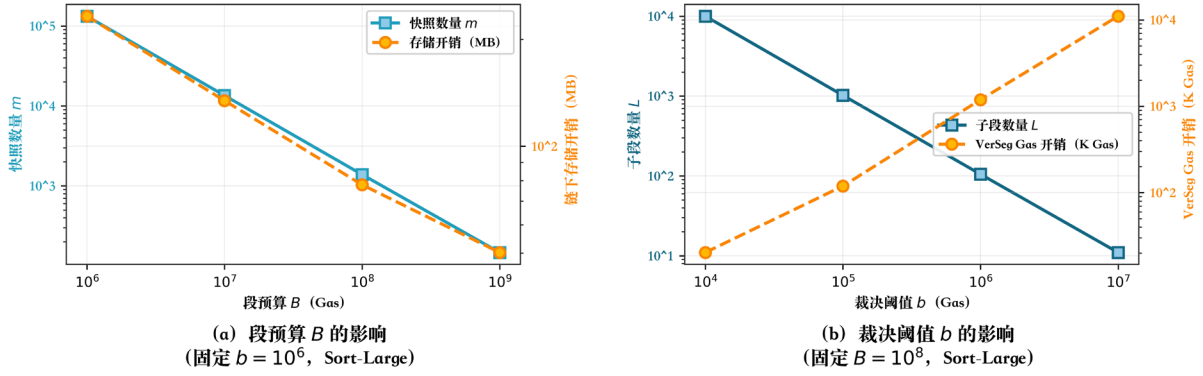


图 9 段预算与裁决阈值对系统开销影响

图 9a 展示了固定裁决阈值 $b = 10^6$ Gas 时，段预算 B 对快照数量与链下存储开销的影响。 B 每增大一个数量级，快照数量 m 相应减少约一个数量级，存储开销同比下降。在默认配置 $B = 10^8$ Gas 下，Sort-Large 任务生成约 1300 个快照，链下存储开销约 61MB，兼顾了伴随式验证的检查粒度与存储可承受性。

图 9b 展示了固定段预算 $B = 10^8$ Gas 时，裁决阈值 b 对二次细分子段数 L 与 VerSeg 链上 Gas 开销的影响。 b 与 L 呈反比关系，与 VerSeg 单次重放 Gas 开销呈正比关系。在默认配置 $b = 10^6$ 下 $L = 100$ 个子段，VerSeg 重放 Gas 开销约为 1.2MGas，占以太坊单笔交易 Gas 上限的 4%，验证了定理 3.2 链上成本有界性在实际系统中的可实现性。 b 的选取本质上是细分承诺序列的链上存储开销与单次裁决重放 Gas 开销之间的权衡。

上述实验验证了可结构化验证的设计目标。自适应切片机制能够在可控的开销下将执行轨迹组织为满足预算约束的段级区间，且 B 与 b 的取值对系统性能的影响符合理论预测，参数可根据应用场景在链下存储成本与链上争议开销之间灵活调节。

3.5.3 争议定位解决效率分析

本小节评估 CleVer 在争议场景下的链上开销表现以及超线性质押机制的博弈效果。

图 10 分别展示了乐观路径与争议路径下五种方案的链上 Gas 开销，以 Sort-Large 任务为测试对象，在任务执行中间位置注入一个状态转移错误触发争议。

可以看到，在乐观路径下图 10a，CleVer 因需在链上提交快照承诺序列，Gas 开销较其他四种方案有一定增长。这是结构化验证所付出的链上承诺提交代价。然而在争议路径下图 10b，CleVer 的优势显著较为明显，在当前测试场景下，其链上争议过程的合计 Gas 开销较其余四种比较方案的平均值降低约 87%。这一优势来源于 CleVer 仅需常数轮次的链上交互。

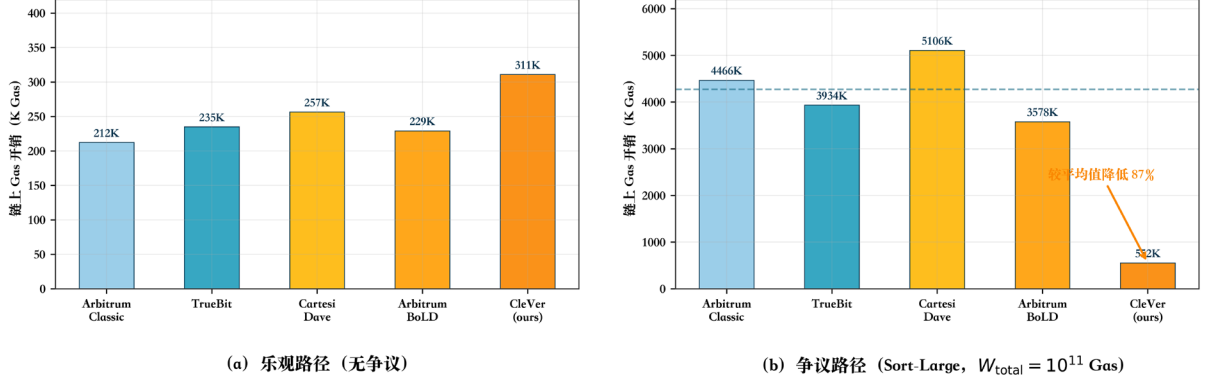


图 10 链上开销对比分析

进一步验证超线性质押机制的博弈效果, 对不同参数设置下的错误方行为进行仿真分析。图 11 展示了质押机制的模拟结果。

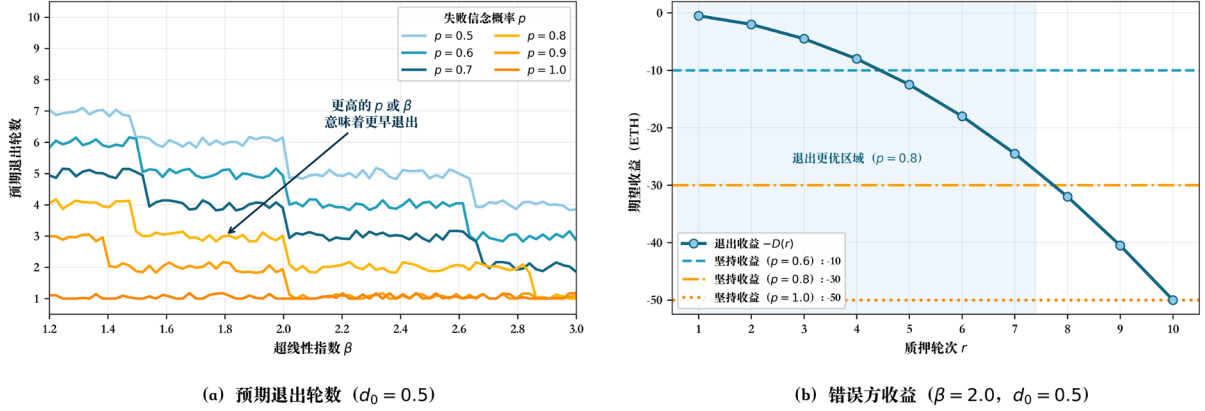


图 11 超线性累计质押博弈效果

图 11a 展示了错误方的期望退出轮次随超线性指数 β 与败诉信念 p 的变化。结果表明, 败诉信念越高, 退出越早, 即使在边界情形下, 退出轮次也随 β 增大而从 7 轮降至 5 轮以内。此外, 超线性指数越大, 退出越早, 因为更高的 β 使边际质押成本增长更快, 压缩了错误方的博弈空间。两个因素协同作用, 使得在 $p \geq 0.7$ 且 $\beta \geq 2$ 的典型配置下, 错误方均在前 2 轮退出, 争议实质上通过经济博弈快速终止, 无需触发链上最终裁决。

图 11b 以二次型质押曲线为例, 展示了错误方在各轮退出的收益与坚持到最终裁决的期望收益对比。退出收益曲线 $-D(r)$ 随轮次单调递减, 而坚持收益为固定值 $(1-2p) \cdot D(g)$ 。绿色区域标示退出收益优于坚持收益的轮次范围。随着轮次推进, 退出代价因 $D(r)$ 的超线性增长而急剧上升, 该结果进一步说明错误方倾向于尽早退出。

综上, 本节实验验证了高效争议定位和有限终止两项设计目标。两层切片机制将争议路径的链上 Gas 开销压缩至与计算规模无关的常数级, 超线性质押机制使错误方在广

泛参数条件下倾向于早期退出，二者协同工作使 CleVer 在争议状态下的链上 Gas 开销约为现有方案的 13%（图中为降低 87%）。

3.5.4 伴随式验证时效性分析

本小节分析伴随式验证相对于事后验证模式在验证路径结构上的时效性优势。所有方案均在统一归一化实验参数下进行比较，度量验证路径结构所导致的相对时延差异，而非生产环境默认协议参数下的绝对时间。其中归一化时间以相同任务在无争议条件下的原始执行时间 T_{exec} 为基准进行度量。

以 DP-Large 任务为对象，在整体执行进度的 40%处注入错误，图 12 展示了各方案从任务开始到错误恢复完成的时间分布。

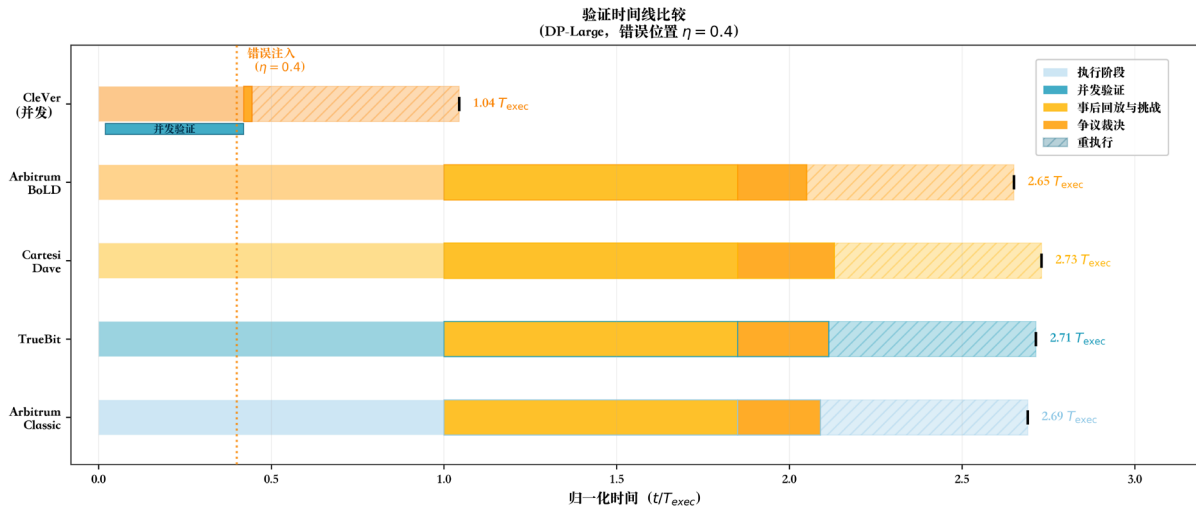


图 12 伴随式验证时间对比分析

四种事后验证方案的验证时间介于 $2.5T_{\text{exec}}$ 至 $3T_{\text{exec}}$ 之间，其中 T_{exec} 为执行时间度量。其中，完整执行阶段和挑战窗口与重放阶段在四者之间相同，差异主要来自争议交互定位阶段，BoLD 凭借层级化争议定位结构在对比方案中争议阶段最短，Cartesi 方案因锦标赛调度开销略长。但这些差异相较于执行与重放阶段并不显著。

如图 12 所示，CleVer 的端到端时间约 $1.04T_{\text{exec}}$ ，较现有方案的验证延迟降低了约 52%。其结构性优势体现在三方面。第一，验证前移，验证与执行并行推进，错误在注入后约 $0.02T_{\text{exec}}$ 即被检测到，无需等待执行完成。第二，消除全量重放阶段，由于验证者在执行过程中已覆盖大部分段的检查，无需在执行结束后再进行独立的全量重放，从路径结构中移除了重放与窗口等待环节。第三，局部纠错，从最近正确快照处接续执行，避免了从任务起始状态全局回退。

3.6 本章小结

本章针对长时间复杂计算任务在链下验证中面临的结构化组织不足、争议处理低效等问题，提出了基于有状态任务切片的链下计算验证方案 CleVer。该方案通过在执行过程中依据段预算约束持续生成中间状态快照，并提交对应的链上承诺，将原本面向全局结果的验证转化为面向局部区间的结构化验证。在此基础上，结合两层切片争议定位与超线性累计质押机制，使争议能够在链下快速收敛、在协议层面有限终止，并在链上以与任务规模解耦的有界开销完成最终裁决。同时，伴随式验证架构将错误发现时点由任务结束后前移至执行过程中，有效降低了验证时延与纠错代价。理论分析表明，CleVer 在裁决正确性、协议终止性和链上成本可控性等方面满足设计要求。实验结果进一步表明，自适应切片机机制引入的额外执行开销不超过 10%，争议路径链上 Gas 开销约为现有方案的 13%，归一化端到端验证时延较事后验证方案降低约 52%。

需要指出的是，本章方案的有效运行仍依赖于验证者诚实履行验证职责。然而，在开放区块链网络环境中，验证者身份暴露以及选举过程可被操纵等因素，会削弱系统中存在诚实验证者这一前提的可靠性。为此，下一章将进一步研究面向链下计算验证场景的验证者安全选举方案。

